

## JQUERY-WEEK3-DAY1-Handson\_Gurunath\_Rageni

### Problem 1 (Level-1): Student Feedback Form Interaction

#### Scenario:

A training institute wants to collect feedback from students through a simple web form. They want to enhance user experience using jQuery to validate inputs and display messages dynamically without reloading the page.

#### Requirements

- Create a feedback form with fields: Name, Email, Course Rating (dropdown), and Comments.
- Use jQuery selectors to capture form inputs.
- On clicking the Submit button, validate that Name and Email are not empty.
- Display a success message dynamically below the form using jQuery DOM manipulation.
- Clear the form fields after successful submission.

#### Technical Constraints

- Use jQuery CDN for integration.
- Use basic jQuery syntax: \$(selector).action().
- Do not use plain JavaScript for validation (only jQuery methods).
- No backend integration required.

#### Expectations:

- Proper use of jQuery selectors and event handling.
- Clean and readable code structure.
- Validation messages should be user-friendly.

#### Code:

```
<!DOCTYPE html>

<html>

<head>

  <title>Student Feedback Form</title>

  <!-- jQuery CDN -->

  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

</head>
```

```
<body>
```

```
<h2>Student Feedback Form</h2>
```

```
<form id="feedbackForm">
```

```
<label>Name:</label><br>
```

```
<input type="text" id="name"><br><br>
```

```
<label>Email:</label><br>
```

```
<input type="email" id="email"><br><br>
```

```
<label>Course Rating:</label><br>
```

```
<select id="rating">
```

```
<option value="">Select Rating</option>
```

```
<option value="Excellent">Excellent</option>
```

```
<option value="Good">Good</option>
```

```
<option value="Average">Average</option>
```

```
</select><br><br>
```

```
<label>Comments:</label><br>
```

```
<textarea id="comments"></textarea><br><br>
```

```
<button type="submit" id="submitBtn">Submit</button>
```

```
</form>
```

```
<!-- Message Area -->
```

```
<div id="message"></div>
```

```
<script>
```

```
$(document).ready(function(){
```

```
$("#feedbackForm").submit(function(event){

    event.preventDefault(); // prevent page reload


    // Get input values using jQuery
    let name = $("#name").val().trim();
    let email = $("#email").val().trim();


    // Validation
    if(name === "" || email === ""){
        $("#message")
            .html("<p style='color:red;'>Please enter both Name and Email.</p>");
    }
    else {
        $("#message")
            .html("<p style='color:green;'>Thank you for your feedback!</p>");

        // Clear form fields
        $("#feedbackForm")[0].reset();
    }

});

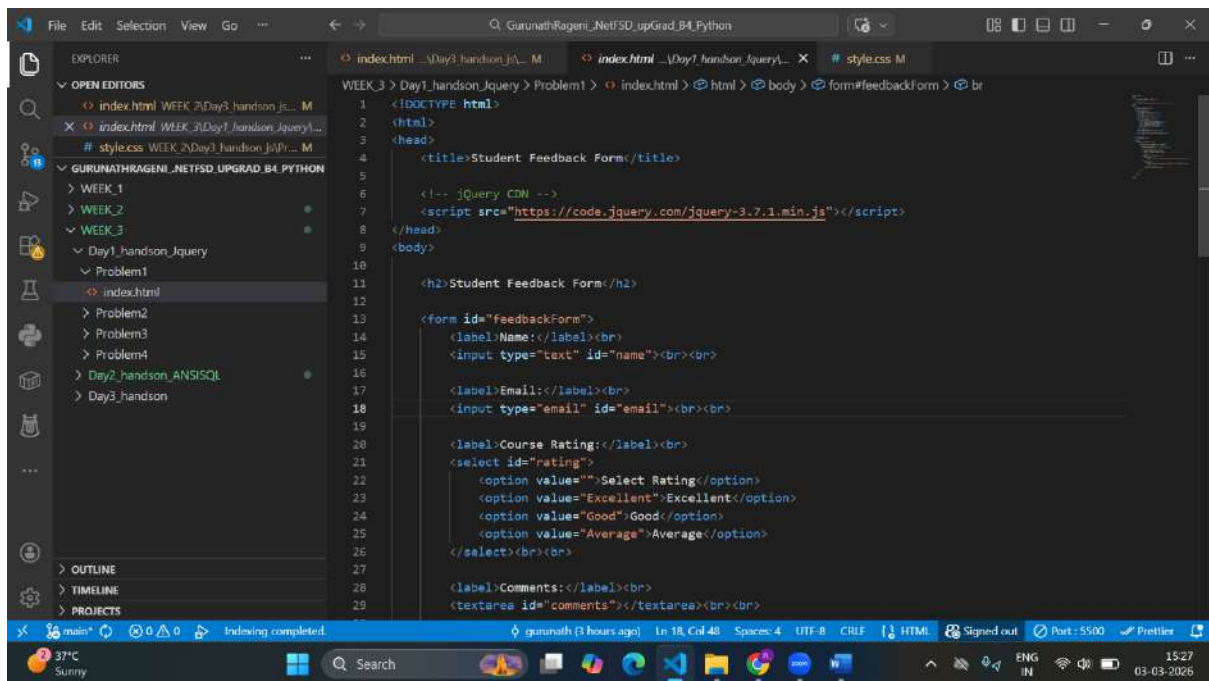
});

</script>

</body>

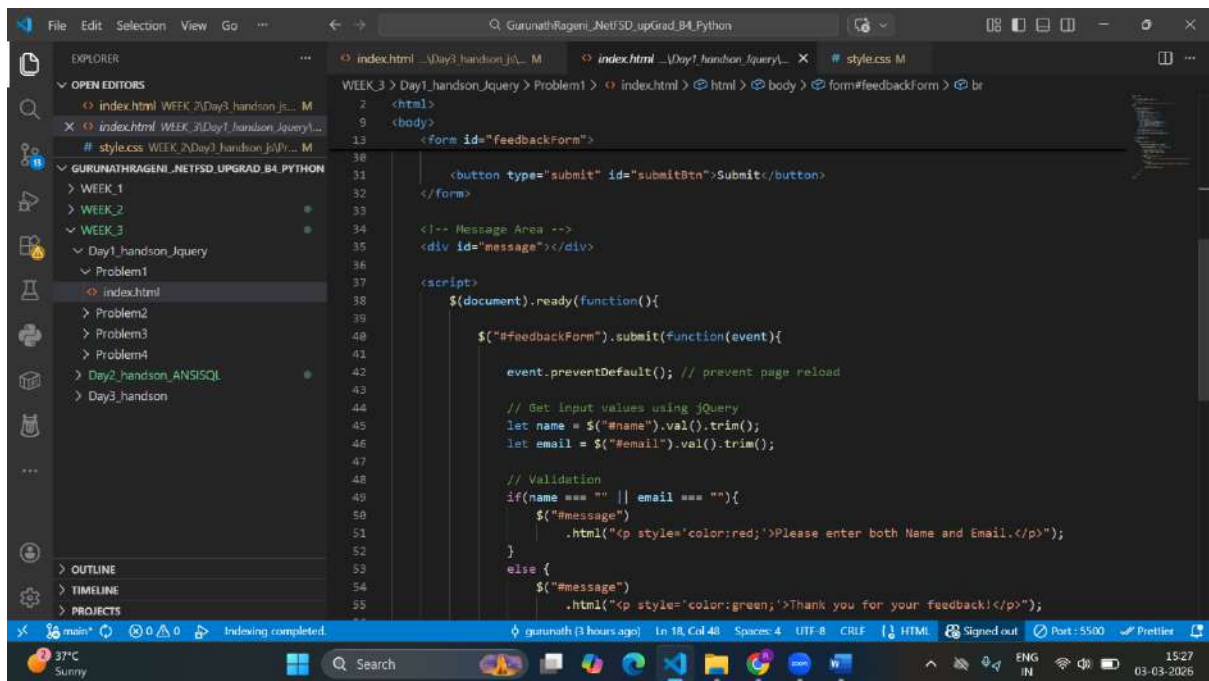
</html>
```

## Code Screenshot:



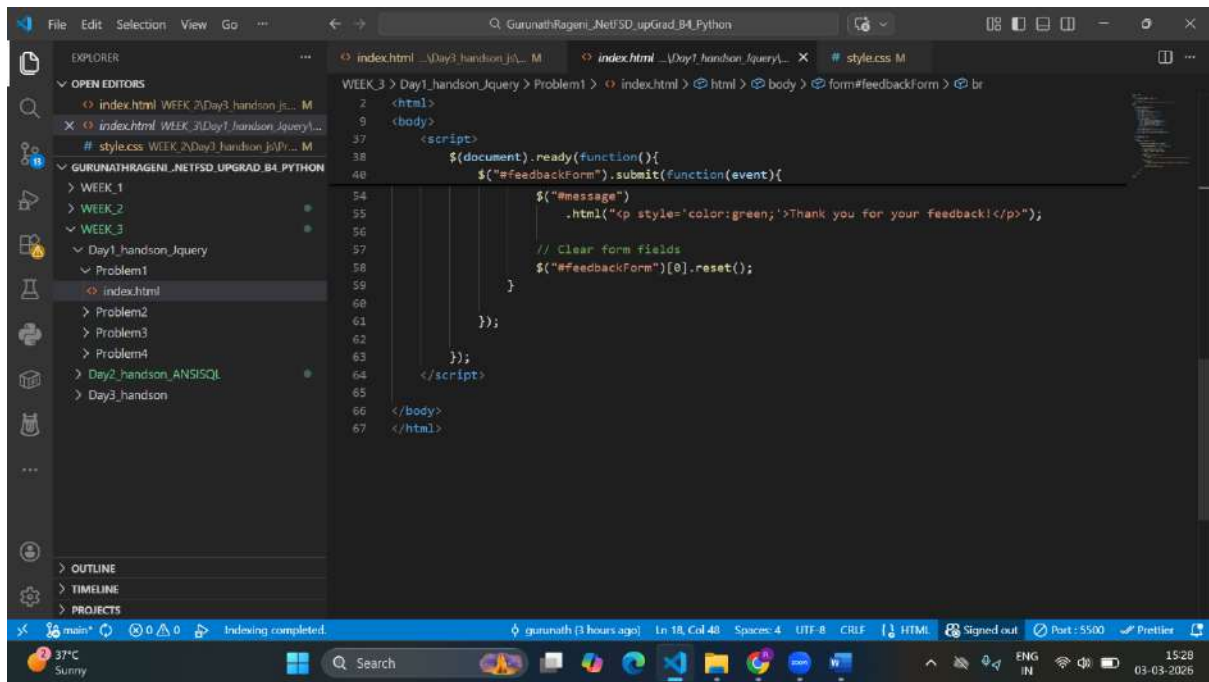
This screenshot shows the VS Code editor with the file explorer on the left displaying a project structure for 'GURUNATHRAGINI.NETFSD.UPGRAD.B4.PYTHON'. The main editor window shows the HTML code for 'index.html' in the 'WEEK\_3 > Day1\_hands\_on\_Jquery > Problem1' directory. The code defines a 'Student Feedback Form' with a title, a jQuery CDN link, and a form with fields for Name, Email, Course Rating (a dropdown menu), and Comments. The form is styled with a blue border and a white background.

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Student Feedback Form</title>
5
6   <!-- jQuery CDN -->
7   <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
8 </head>
9 <body>
10
11   <h2>Student Feedback Form</h2>
12
13   <form id="feedbackForm">
14     <label>Name:</label><br>
15     <input type="text" id="name"><br>
16
17     <label>Email:</label><br>
18     <input type="email" id="email"><br>
19
20     <label>Course Rating:</label><br>
21     <select id="rating">
22       <option value="">Select Rating</option>
23       <option value="Excellent">Excellent</option>
24       <option value="Good">Good</option>
25       <option value="Average">Average</option>
26     </select><br>
27
28     <label>Comments:</label><br>
29     <textarea id="comments"></textarea><br>
```

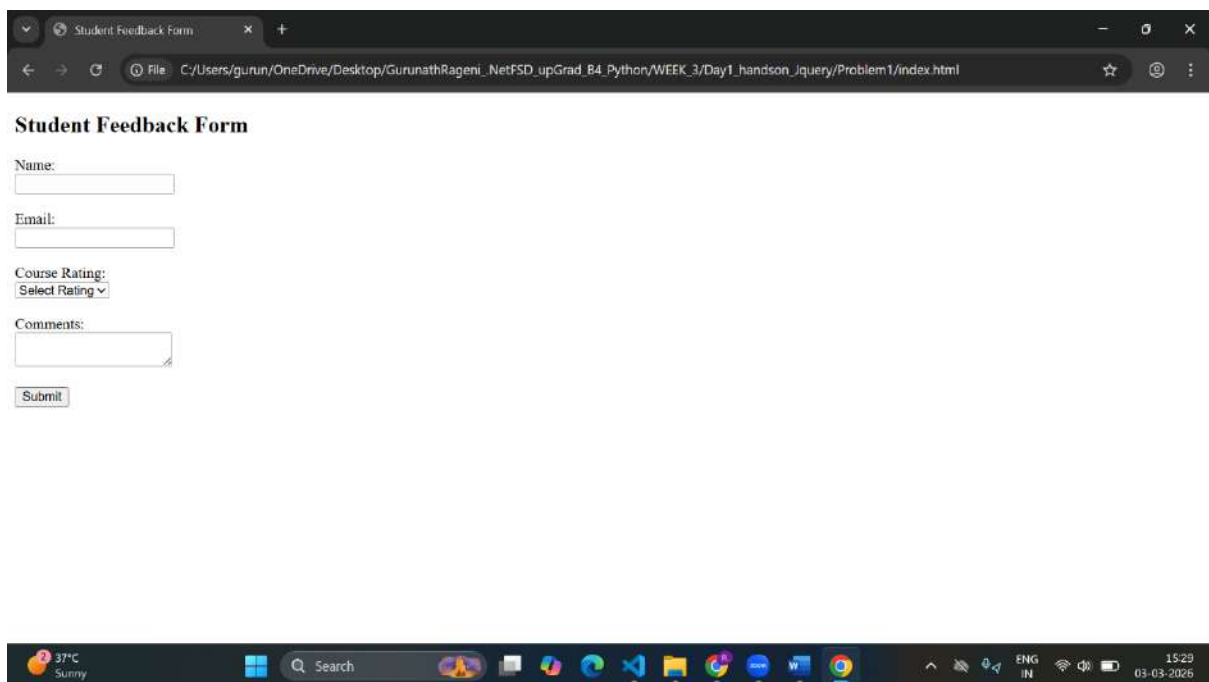


This screenshot shows the VS Code editor with the file explorer on the left displaying the same project structure. The main editor window shows the JavaScript code for 'index.html' in the 'WEEK\_3 > Day1\_hands\_on\_Jquery > Problem1' directory. The code includes a submit button, a message area, and a jQuery script that handles the form submission. The script uses jQuery to get the values of the name and email fields, performs validation, and displays a message in the message area.

```
30   <button type="submit" id="submitBtn">Submit</button>
31 </form>
32
33   <!-- Message Area -->
34   <div id="message"></div>
35
36   <script>
37     $(document).ready(function(){
38
39       $("#feedbackForm").submit(function(event){
40
41         event.preventDefault(); // prevent page reload
42
43         // Get input values using jQuery
44         let name = $("#name").val().trim();
45         let email = $("#email").val().trim();
46
47         // Validation
48         if(name === "" || email === ""){
49           $("#message")
50             .html("<p style='color:red;'>Please enter both Name and Email.</p>");
51         }
52         else {
53           $("#message")
54             .html("<p style='color:green;'>Thank you for your feedback!</p>");
55         }
56       });
57     });
58   </script>
```



Output Screenshot:



Explanation:

This application uses **jQuery** to handle form submission and DOM manipulation without reloading the page by using `event.preventDefault()`. Input values are accessed using jQuery selectors (`$("#id")`) and validated to ensure required fields are not empty.

Based on validation, dynamic feedback messages are inserted using .html() to update the DOM instantly. The form is reset using reset() after successful submission, improving user experience through interactive client-side validation.

## Problem 2 (Level-1): Dynamic FAQ Toggle Section

### Scenario:

A website contains a Frequently Asked Questions (FAQ) section. The company wants answers to appear and disappear when users click on the questions.

### Requirements

- Create at least 5 FAQ questions with hidden answers.
- Use jQuery click() event to toggle answers.
- Apply slideToggle() or show()/hide() methods.
- Highlight the selected question using CSS class manipulation.

### Technical Constraints

- Use only jQuery methods for DOM manipulation.
- Use class selectors and ID selectors appropriately.
- Avoid inline JavaScript.

### Expectations:

- Smooth UI interaction.
- Proper use of jQuery effects.
- Organized HTML structure.

### Code:

```
<!DOCTYPE html>

<html>

<head>

  <title>FAQ Toggle</title>

  <meta name="viewport" content="width=device-width, initial-scale=1">


  <!-- jQuery CDN -->
```

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
```

```
<style>
```

```
body{  
    font-family: Arial, sans-serif;  
    padding: 40px;  
    background: #f4f4f4;  
}
```

```
h2{  
    text-align: center;  
    margin-bottom: 30px;  
}
```

```
.faq-container{  
    max-width: 700px;  
    margin: auto;  
}
```

```
.faq-item {  
    background: white;  
    margin-bottom: 10px;  
    border-radius: 5px;  
    overflow: hidden;  
}
```

```
.question {  
    padding: 15px;  
    cursor: pointer;  
    font-weight: bold;  
    background: #007bff;
```

```
        color: white;
    }

    .question.active {
        background: #0056b3;
    }

    .answer {
        padding: 15px;
        display: none;
        background: #f9f9f9;
    }
</style>
</head>

<body>

<h2>Frequently Asked Questions</h2>

<div class="faq-container">

    <div class="faq-item">
        <div class="question">What is your return policy?</div>
        <div class="answer">You can return products within 30 days of purchase.</div>
    </div>

    <div class="faq-item">
        <div class="question">Do you offer international shipping?</div>
        <div class="answer">Yes, we ship to selected countries worldwide.</div>
    </div>

</div>
```



```
<div class="faq-item">
  <div class="question">How can I track my order?</div>
  <div class="answer">You will receive a tracking link via email after dispatch.</div>
</div>
```

```
<div class="faq-item">
  <div class="question">What payment methods are accepted?</div>
  <div class="answer">We accept credit cards, debit cards, and UPI.</div>
</div>
```

```
<div class="faq-item">
  <div class="question">How can I contact support?</div>
  <div class="answer">You can contact us via email or phone support.</div>
</div>
```

```
</div>
```

```
<script>
```

```
$(document).ready(function () {
```

```
  $(".question").click(function () {
```

```
    // Close other answers
```

```
    $(".answer").not($(this).next()).slideUp();
```

```
    $(".question").not($(this)).removeClass("active");
```

```
    // Toggle clicked answer
```

```
    $(this).next(".answer").slideToggle();
```

```
    // Highlight active question
```

```
    $(this).toggleClass("active");
```

});

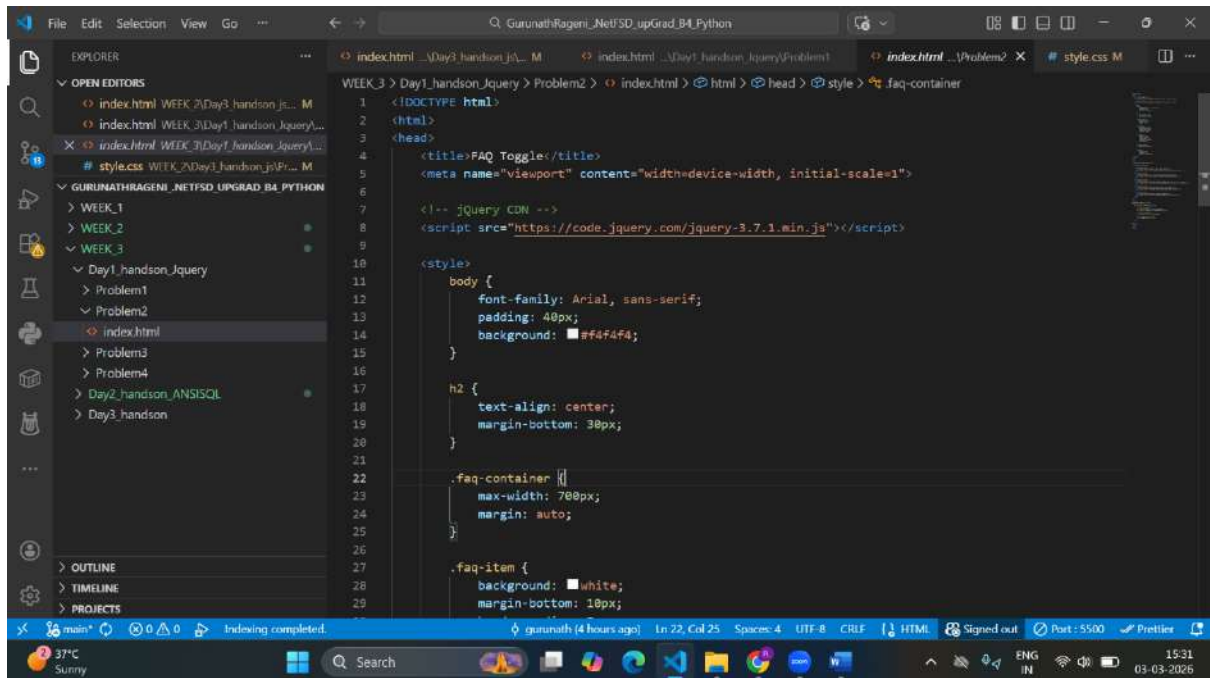
});

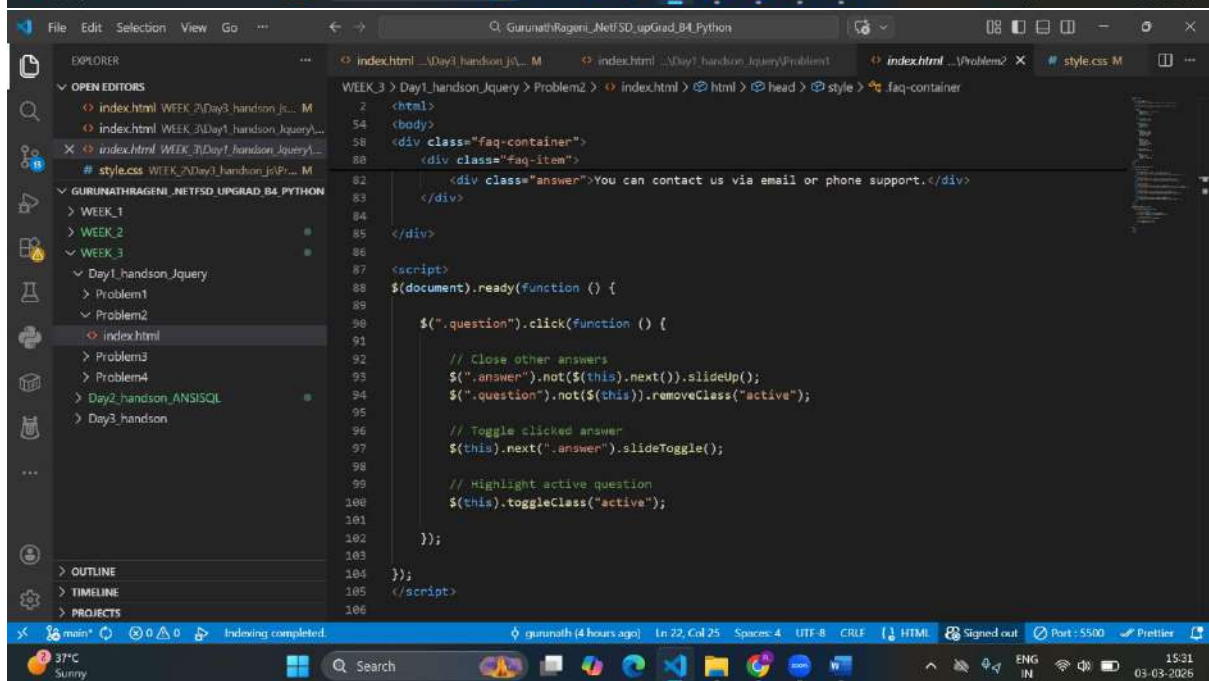
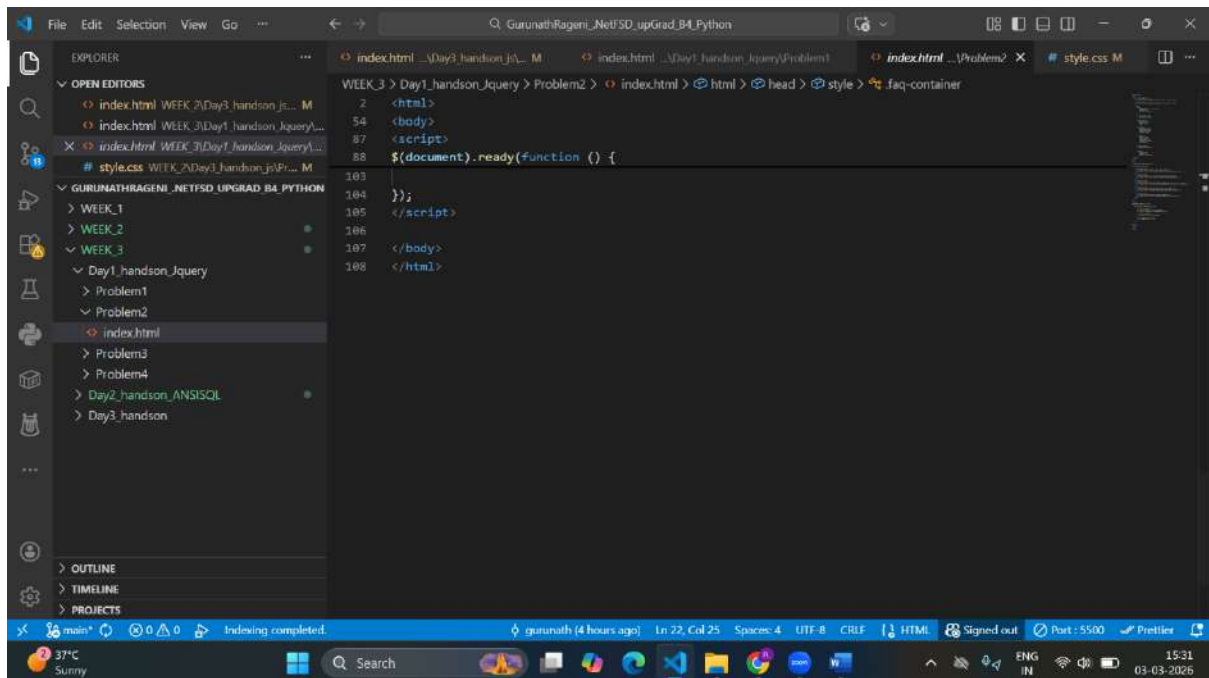
</script>

</body>

</html>

Code Screenshots:





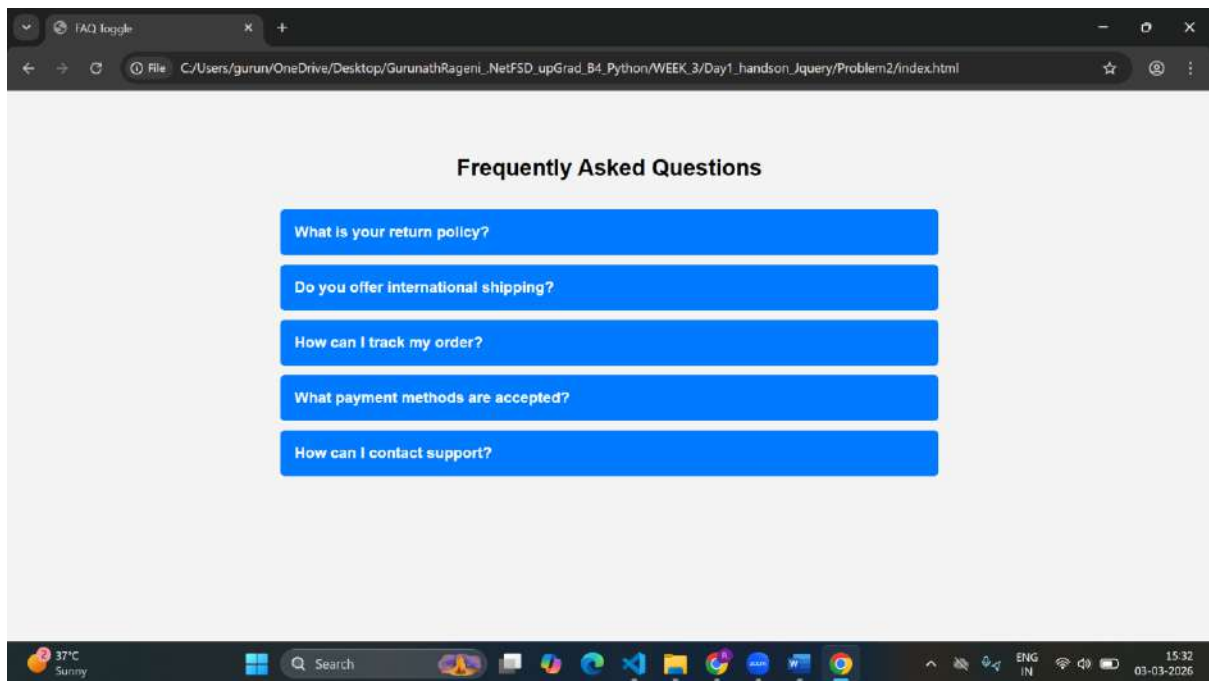
This screenshot shows the VS Code editor with the Explorer sidebar on the left displaying a project structure for 'GURUNATHRAGANI\_NETFSD\_UPGRAD\_B4\_PYTHON'. The main editor area shows the 'index.html' file for 'WEEK\_3 > Day1\_handson\_Jquery > Problem2'. The code defines an FAQ container with a heading and several question-answer pairs.

```
2 <html>
3 <head>
4 <title>
5 </title>
6 </head>
7 <body>
8
9 <h2>Frequently Asked Questions</h2>
10
11 <div class="faq-container">
12
13   <div class="faq-item">
14     <div class="question">What is your return policy?</div>
15     <div class="answer">You can return products within 30 days of purchase.</div>
16   </div>
17
18   <div class="faq-item">
19     <div class="question">Do you offer international shipping?</div>
20     <div class="answer">Yes, we ship to selected countries worldwide.</div>
21   </div>
22
23   <div class="faq-item">
24     <div class="question">How can I track my order?</div>
25     <div class="answer">You will receive a tracking link via email after dispatch.</div>
26   </div>
27
28   <div class="faq-item">
29     <div class="question">What payment methods are accepted?</div>
30     <div class="answer">We accept credit cards, debit cards, and UPI.</div>
31   </div>
32
33   <div class="faq-item">
34     <div class="question">How can I contact support?</div>
35   </div>
36 </div>
37 </body>
38 </html>
```

This screenshot shows the VS Code editor with the same project structure as the first image. The main editor area shows the 'style.css' file for 'WEEK\_3 > Day1\_handson\_Jquery > Problem2'. The code defines CSS styles for the FAQ container, including a base style for the .faq-item class and specific styles for the .question and .answer elements.

```
1 <!-- Base styles for FAQ items -->
2 .faq-item {
3   border-radius: 5px;
4   overflow: hidden;
5 }
6
7 <!-- Question styles -->
8 .question {
9   padding: 15px;
10  cursor: pointer;
11  font-weight: bold;
12  background: #e0e0e0;
13  color: white;
14 }
15
16 <!-- Active question styles -->
17 .question.active {
18   background: #000000;
19 }
20
21 <!-- Answer styles -->
22 .answer {
23   padding: 15px;
24   display: none;
25   background: #f9f9f9;
26 }
27
28 </style>
29 </head>
30 <body>
```

## Output Screenshot:



## Explanation:

This FAQ application uses **jQuery** to implement an interactive accordion-style toggle effect with smooth animations using `slideToggle()` and `slideUp()`. When a question is clicked, it dynamically shows the corresponding answer while closing others, ensuring only one section remains open at a time.

DOM traversal methods like `$(this).next()` and filtering with `.not()` are used to control related elements efficiently. CSS classes are dynamically added and removed (`toggleClass`, `removeClass`) to visually highlight the active question, improving user experience and UI clarity.

## Problem 3 (Level-2): Product List with Add to Cart Counter

### Scenario:

An e-commerce website wants to display a list of products. When users click 'Add to Cart', the cart counter at the top should increase dynamically.

### Requirements

- Display at least 4 products with 'Add to Cart' buttons.
- Use jQuery to handle button click events.
- Increase the cart counter dynamically without page refresh.
- Disable the button once the product is added.
- Display a small confirmation message beside the product.

## 🔗 Technical Constraints

- Use jQuery event delegation if needed.
- Use .text(), .attr(), and .prop() methods.
- No backend or database integration.

## Expectations:

- Efficient event handling.
- Proper use of jQuery methods for attribute manipulation.
- Clean UI updates.

## Code:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Product Cart Counter</title>
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1">
```

```
  <!-- jQuery CDN -->
```

```
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
```

```
  <style>
```

```
    body {  
      font-family: Arial, sans-serif;  
      padding: 40px;  
      background: #f4f4f4;  
    }
```

```
    .header {  
      display: flex;  
      justify-content: space-between;
```

```
    align-items: center;
    margin-bottom: 30px;
}
```

```
.cart {
    background: #007bff;
    color: white;
    padding: 8px 15px;
    border-radius: 20px;
}
```

```
.product {
    background: white;
    padding: 15px;
    margin-bottom: 15px;
    border-radius: 5px;
}
```

```
.product button {
    margin-top: 10px;
}
```

```
.confirmation {
    color: green;
    font-size: 14px;
    margin-left: 10px;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="header">

  <h2>Our Products</h2>

  <div class="cart">

    Cart: <span id="cart-count">0</span>

  </div>

</div>
```

```
<div id="product-list">

  <div class="product">

    <h4>Laptop</h4>

    <button class="add-btn">Add to Cart</button>

    <span class="confirmation"></span>

  </div>
```

```
<div class="product">

  <h4>Smartphone</h4>

  <button class="add-btn">Add to Cart</button>

  <span class="confirmation"></span>

</div>
```

```
<div class="product">

  <h4>Headphones</h4>

  <button class="add-btn">Add to Cart</button>

  <span class="confirmation"></span>

</div>
```

```
<div class="product">

  <h4>Smartwatch</h4>

  <button class="add-btn">Add to Cart</button>
```



```
<span class="confirmation"></span>
```

```
</div>
```

```
</div>
```

```
<script>
```

```
$(document).ready(function () {
```

```
    let cartCount = 0;
```

```
    // Event Delegation
```

```
    $("#product-list").on("click", ".add-btn", function () {
```

```
        cartCount++;
```

```
        // Update counter using .text()
```

```
        $("#cart-count").text(cartCount);
```

```
        // Disable button using .prop()
```

```
        $(this).prop("disabled", true);
```

```
        // Change button text using .text()
```

```
        $(this).text("Added");
```

```
        // Add attribute using .attr()
```

```
        $(this).attr("data-added", "true");
```

```
        // Show confirmation message
```

```
        $(this).siblings(".confirmation")
```

```
            .text("✓ Added to cart!");
```

});

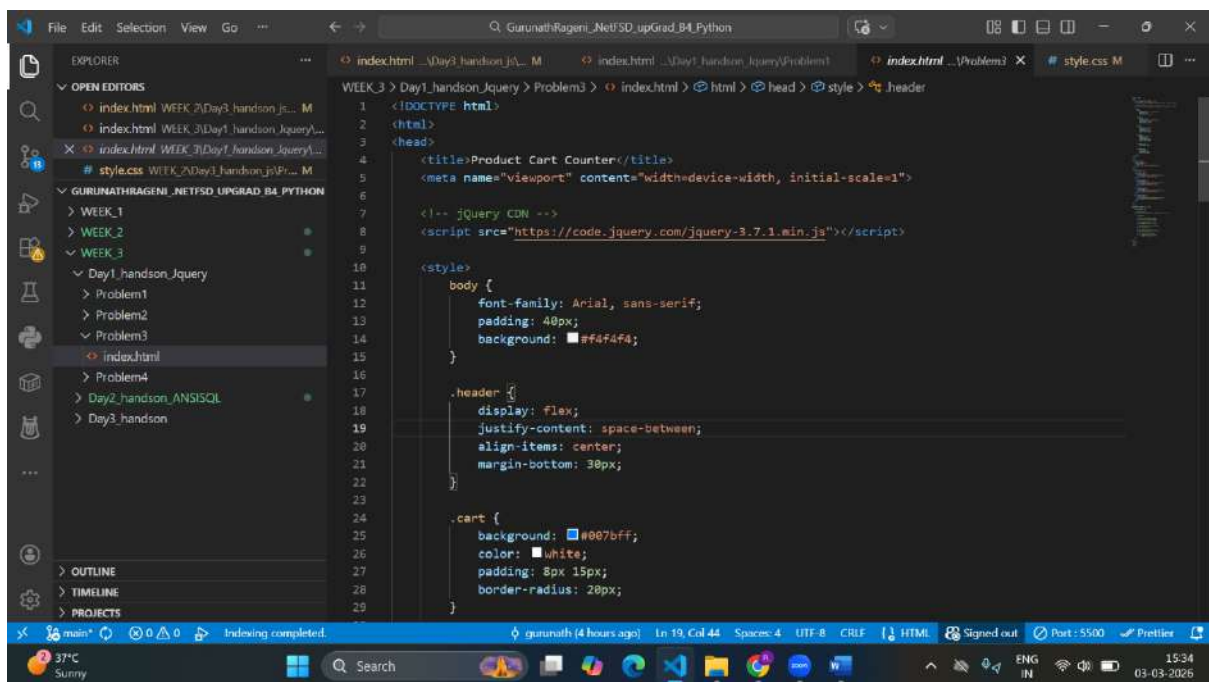
});

</script>

</body>

</html>

Code ScreenShots:



This screenshot shows the VS Code editor with the following details:

- Explorer:** The file tree on the left shows a project structure with folders for WEEK\_1, WEEK\_2, WEEK\_3, and Day1\_handson\_Jquery. The 'index.html' file is selected under Day1\_handson\_Jquery.
- Editor:** The main editor area displays the CSS code for 'style.css'. The code defines styles for a product list, including a header, product items, a confirmation message, and a cart count.
- Code:**

```
WEEK_3 > Day1_handson_Jquery > Problem3 > index.html > html > head > style > .header
2 <html>
3 <head>
18 <style>
30
31 .product {
32   background: white;
33   padding: 15px;
34   margin-bottom: 15px;
35   border-radius: 5px;
36 }
37
38 .product button {
39   margin-top: 10px;
40 }
41
42 .confirmation {
43   color: green;
44   font-size: 14px;
45   margin-left: 10px;
46 }
47 </style>
48 </head>
49
50 <body>
51
52 <div class="header">
53   <h2>Our Products</h2>
54   <div class="cart">
55     Cart: <span id="cart-count">0</span>
```
- Status Bar:** The bottom status bar shows the file is indexed, the editor is in UTF-8 encoding, and the port is 5500.

This screenshot shows the VS Code editor with the following details:

- Explorer:** The file tree on the left shows a project structure with folders for WEEK\_1, WEEK\_2, WEEK\_3, and Day1\_handson\_Jquery. The 'index.html' file is selected under Day1\_handson\_Jquery.
- Editor:** The main editor area displays the JavaScript code for 'index.html'. The code includes HTML structure for a product list and a JavaScript script to handle the 'Add to Cart' button click.
- Code:**

```
WEEK_3 > Day1_handson_Jquery > Problem3 > index.html > html > head > style > .header
2 <html>
50 <body>
59 <div id="product-list">
79   <div class="product">
81     <button class="add-btn">Add to Cart</button>
82     <span class="confirmation"></span>
83   </div>
84
85 </div>
86
87 <script>
88 $(document).ready(function () {
89
90   let cartCount = 0;
91
92   // Event Delegation
93   $("#product-list").on("click", ".add-btn", function () {
94
95     cartCount++;
96
97     // Update counter using .text()
98     $("#cart-count").text(cartCount);
99
100    // Disable button using .prop()
101    $(this).prop("disabled", true);
102
103    // Change button text using .text()
104    $(this).text("Added");
105  });
106 }
```
- Status Bar:** The bottom status bar shows the file is indexed, the editor is in UTF-8 encoding, and the port is 5500.

File Edit Selection View Go ...

Q GurunathRagani\_Net/SD\_upGrad\_B4\_Python

EXPLORER

OPEN EDITORS

index.html WEEK\_2/Day3\_handson.js... M

index.html WEEK\_3/Day1\_handson JQuery... M

index.html WEEK\_3/Day1\_handson JQuery... M

style.css WEEK\_2/Day1\_handson.jsPr... M

GURUNATHRAGANI\_NET/SD UPGRAD B4 PYTHON

WEEK\_1

WEEK\_2

WEEK\_3

Day1\_handson JQuery

Problem1

Problem2

Problem3

index.html

Problem4

Day2\_handson\_ANSISQL

Day3\_handson

OUTLINE

TIMELINE

PROJECTS

index.html .../Day3\_handson.js... M

index.html .../Day1\_handson JQuery/Problem1

index.html .../Problem3 X

style.css M

WEEK\_3 > Day1\_handson JQuery > Problem3 > index.html > html > head > style > .header

```
2 <html>
50 <body>
87 <script>
88 $(document).ready(function () {
93   $("#product-list").on("click", ".add-btn", function () {
105
106     // Add attribute using .attr()
107     $(this).attr("data-added", "true");
108
109     // Show confirmation message
110     $(this).siblings(".confirmation")
111       .text("✓ Added to cart!");
112
113   });
114
115 });
116 </script>
117
118 </body>
119 </html>
```

main\* 0 0 Indexing completed. gurunath (4 hours ago) Ln 19, Col 44 Spaces: 4 UTF-8 CRLF HTML Signed out Port: 5500 Prettier

37°C Sunny

File Edit Selection View Go ...

Q GurunathRagani\_Net/SD\_upGrad\_B4\_Python

EXPLORER

OPEN EDITORS

index.html WEEK\_2/Day3\_handson.js... M

index.html WEEK\_3/Day1\_handson JQuery... M

index.html WEEK\_3/Day1\_handson JQuery... M

style.css WEEK\_2/Day1\_handson.jsPr... M

GURUNATHRAGANI\_NET/SD UPGRAD B4 PYTHON

WEEK\_1

WEEK\_2

WEEK\_3

Day1\_handson JQuery

Problem1

Problem2

Problem3

index.html

Problem4

Day2\_handson\_ANSISQL

Day3\_handson

OUTLINE

TIMELINE

PROJECTS

index.html .../Day3\_handson.js... M

index.html .../Day1\_handson JQuery/Problem1

index.html .../Problem3 X

style.css M

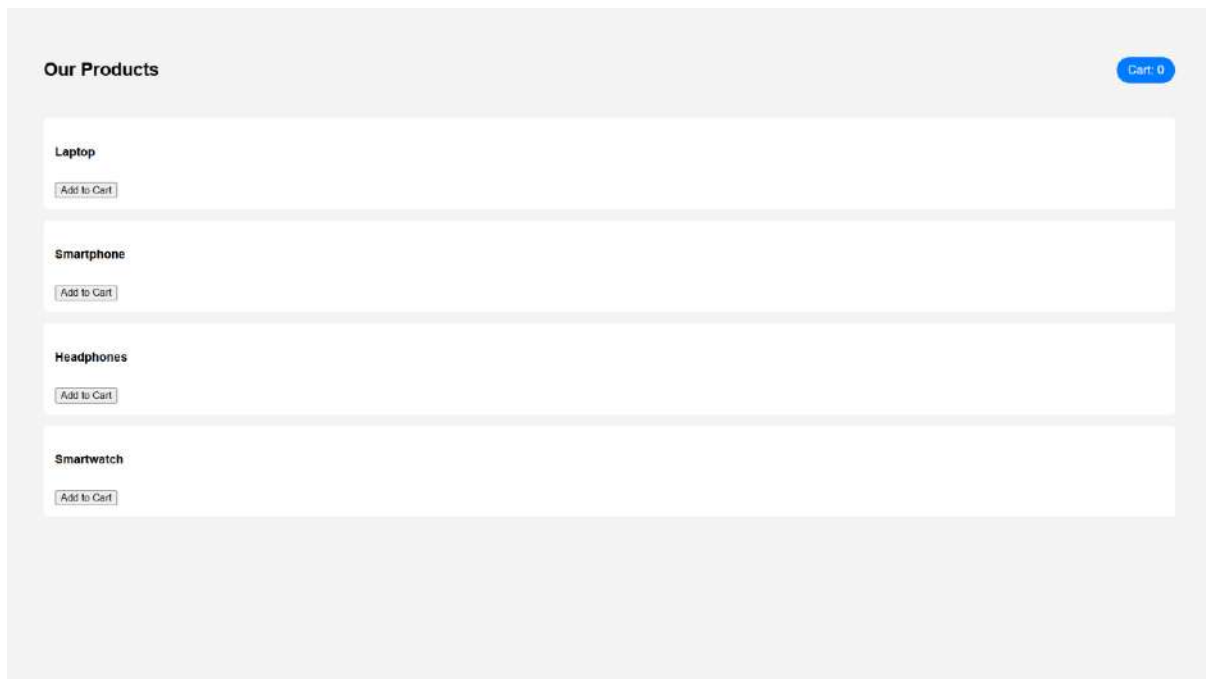
WEEK\_3 > Day1\_handson JQuery > Problem3 > index.html > html > head > style > .header

```
2 <html>
50 <body>
52 <div class="header">
54 <div class="cart">
56 </div>
57 </div>
58
59 <div id="product-list">
60
61 <div class="product">
62 <h4>Laptop</h4>
63 <button class="add-btn">Add to Cart</button>
64 <span class="confirmation"></span>
65 </div>
66
67 <div class="product">
68 <h4>Smartphone</h4>
69 <button class="add-btn">Add to Cart</button>
70 <span class="confirmation"></span>
71 </div>
72
73 <div class="product">
74 <h4>Headphones</h4>
75 <button class="add-btn">Add to Cart</button>
76 <span class="confirmation"></span>
77 </div>
78
79 <div class="product">
80 <h4>Smartwatch</h4>
```

main\* 0 0 Indexing completed. gurunath (4 hours ago) Ln 19, Col 44 Spaces: 4 UTF-8 CRLF HTML Signed out Port: 5500 Prettier

37°C Sunny

## Output Screenshot:



## Explanation:

This application uses **jQuery event delegation** by attaching a single click listener to `#product-list`, allowing dynamically handled `.add-btn` elements to update the cart counter efficiently. The cart value is incremented and updated in real time using `.text()` to reflect changes in the DOM.

It demonstrates DOM manipulation methods like `.prop()` to disable buttons, `.attr()` to add custom attributes, and `.siblings()` to target related elements for confirmation messages. This approach ensures clean, scalable interaction handling without adding multiple event listeners.

## Problem 4 (Level-2): Interactive To-Do List Application

### Scenario:

A productivity web app requires a dynamic To-Do List where users can add, mark as complete, and remove tasks using jQuery.

### Requirements

- Provide an input field and Add button to insert tasks.
- Display tasks dynamically in a list.
- Allow users to mark tasks as completed (strike-through effect).

- Provide a delete button for each task.
- Display total task count and completed task count dynamically.

#### 📋 Technical Constraints

- Use jQuery for all DOM manipulation and event handling.
- Use append(), remove(), toggleClass(), and on() methods.
- No external libraries except jQuery.

#### Expectations:

- Proper dynamic element handling.
- Efficient event binding for dynamically created elements.
- Responsive and interactive UI behavior.

#### Code:

```
<!DOCTYPE html>

<html>

<head>

  <title>jQuery To-Do App</title>

  <meta name="viewport" content="width=device-width, initial-scale=1">

  <!-- jQuery CDN -->

  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

  <style>

    body {

      font-family: Arial, sans-serif;

      padding: 40px;

      background: #f4f4f4;

    }

    .container {
```

```
    max-width: 500px;
    margin: auto;
    background: white;
    padding: 20px;
    border-radius: 8px;
}
```

```
.task {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 8px;
    margin-top: 8px;
    background: #f9f9f9;
    border-radius: 4px;
}
```

```
.completed {
    text-decoration: line-through;
    color: gray;
}
```

```
button {
    cursor: pointer;
}
```

```
.stats {
    margin-top: 15px;
    font-size: 14px;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

```
  <h2>To-Do List</h2>
```

```
  <input type="text" id="taskInput" placeholder="Enter a task">
```

```
  <button id="addBtn">Add</button>
```

```
  <div id="taskList"></div>
```

```
  <div class="stats">
```

```
    Total Tasks: <span id="totalCount">0</span> |
```

```
    Completed: <span id="completedCount">0</span>
```

```
  </div>
```

```
</div>
```

```
<script>
```

```
$(document).ready(function () {
```

```
  let totalTasks = 0;
```

```
  let completedTasks = 0;
```

```
  // Function to update counters
```

```
  function updateCounts() {
```

```
    $("#totalCount").text(totalTasks);
```

```
    $("#completedCount").text(completedTasks);
```

```
  }
```

```
  // Add Task
```



```

$("#addBtn").on("click", function () {

    const taskText = $("#taskInput").val().trim();

    if (taskText === "") return;

    totalTasks++;

    const taskItem = `
        <div class="task">
            <span class="task-text">${taskText}</span>
            <div>
                <button class="completeBtn">✓</button>
                <button class="deleteBtn">✗</button>
            </div>
        </div>
    `;

    $("#taskList").append(taskItem);

    $("#taskInput").val("");

    updateCounts();

});

// Mark as Complete (Event Delegation)
$("#taskList").on("click", ".completeBtn", function () {

    const taskText = $(this).closest(".task").find(".task-text");

    taskText.toggleClass("completed");

    if (taskText.hasClass("completed")) {

```

```
        completedTasks++;
    } else {
        completedTasks--;
    }

    updateCounts();
});

// Delete Task (Event Delegation)
$("#taskList").on("click", ".deleteBtn", function () {

    const taskText = $(this).closest(".task").find(".task-text");

    if (taskText.hasClass("completed")) {
        completedTasks--;
    }

    totalTasks--;

    $(this).closest(".task").remove();

    updateCounts();
});

});
</script>

</body>
</html>
```

## Code Screenshots:

This screenshot shows the initial setup of a web application in VS Code. The Explorer panel on the left shows a project structure with folders for WEEK\_1, WEEK\_2, WEEK\_3, and Day1\_handson\_Jquery. The main editor displays the HTML file 'index.html' for 'Day1\_handson\_Jquery > Problem4'. The code includes a DOCTYPE declaration, HTML and head tags, a title 'jQuery To-Do App', a viewport meta tag, a jQuery CDN script, and CSS styles for the body and a container. The body has a dark blue background, and the container is white with a border and padding.

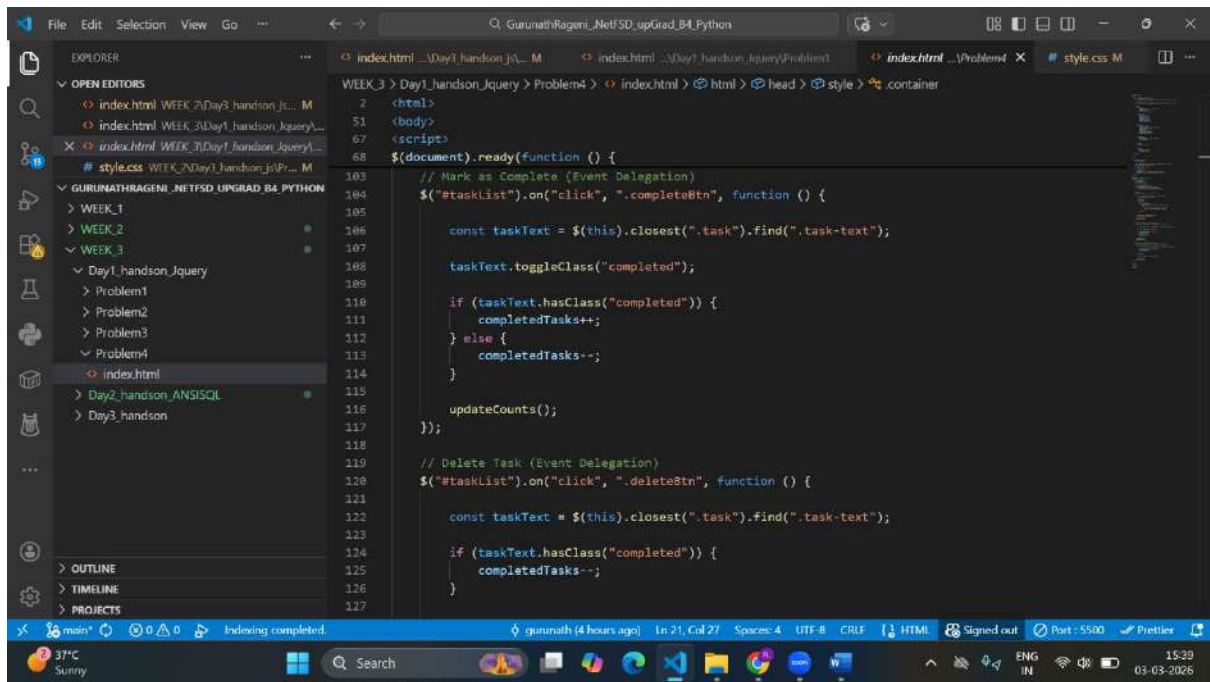
```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>jQuery To-Do App</title>
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6
7   <!-- jQuery CDN -->
8   <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
9
10  <style>
11    body {
12      font-family: Arial, sans-serif;
13      padding: 40px;
14      background: #444f4f;
15    }
16
17    .container {
18      max-width: 500px;
19      margin: auto;
20      background: white;
21      padding: 20px;
22      border-radius: 8px;
23    }
24
25    .task {
26      display: flex;
27      justify-content: space-between;
28      align-items: center;
29      padding: 8px;
```

This screenshot shows the continuation of the HTML and CSS code from the previous screenshot. The CSS styles for the .task, .completed, button, and .stats classes are defined. The HTML body contains a container div with a class attribute, and the head is closed. The status bar at the bottom indicates the file is 'index.html' and the project is 'GurunathiRagani\_NetFSUpGrad\_B4\_Python'.

```
30      margin-top: 8px;
31      background: #f9f9f9;
32      border-radius: 4px;
33    }
34
35    .completed {
36      text-decoration: line-through;
37      color: gray;
38    }
39
40    button {
41      cursor: pointer;
42    }
43
44    .stats {
45      margin-top: 15px;
46      font-size: 14px;
47    }
48  </style>
49 </head>
50
51 <body>
52
53 <div class="container">
54   <h2>To-Do List</h2>
```

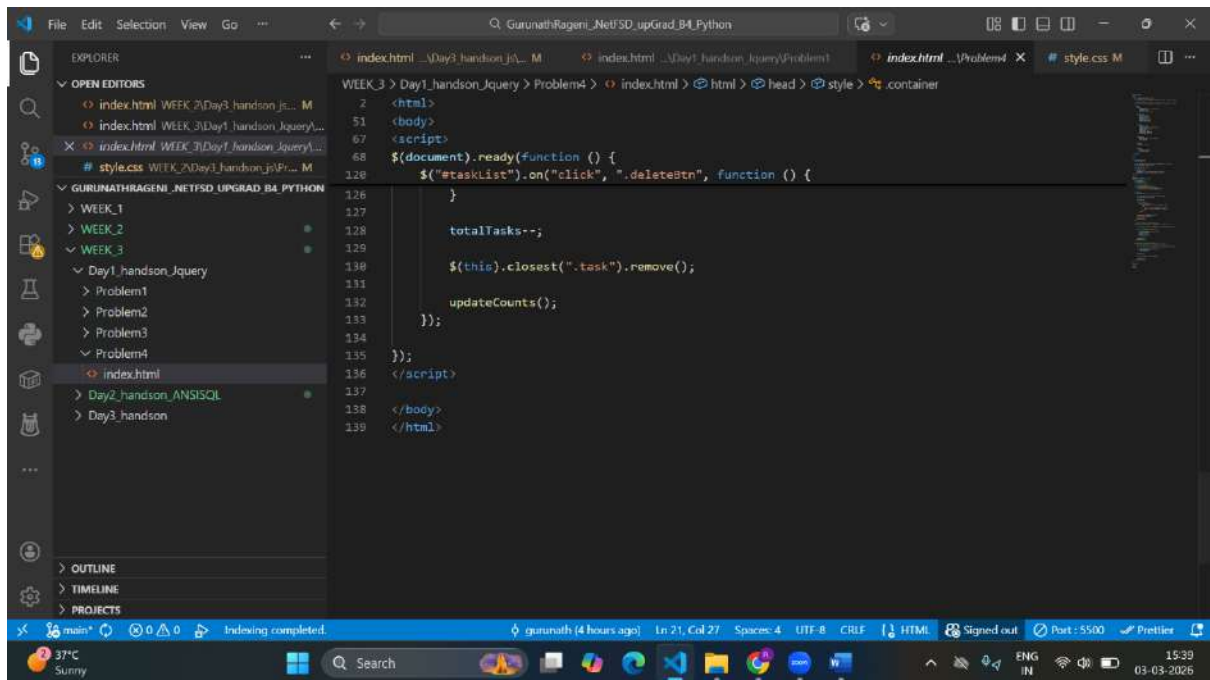
```
index.html WEEK_3 > Day1_handson_Jquery > Problem4 > index.html > html > head > style > .container
2 <html>
51 <body>
53 <div class="container">
54 <h2>To-Do List</h2>
55
56 <input type="text" id="taskInput" placeholder="Enter a task">
57 <button id="addBtn">Add</button>
58
59 <div id="taskList"></div>
60
61 <div class="stats">
62 Total Tasks: <span id="totalCount">0</span> |
63 Completed: <span id="completedCount">0</span>
64 </div>
65 </div>
66
67 <script>
68 $(document).ready(function () {
69
70 let totalTasks = 0;
71 let completedTasks = 0;
72
73 // Function to update counters
74 function updateCounts() {
75 $("#totalCount").text(totalTasks);
76 $("#completedCount").text(completedTasks);
77 }
78
79 // Add Task
```

```
79 // Add Task
80 $("#addBtn").on("click", function () {
81
82 const taskText = $("#taskInput").val().trim();
83 if (taskText === "") return;
84
85 totalTasks++;
86
87 const taskItem = `
88 <div class="task">
89 <span class="task-text">${taskText}</span>
90 <div>
91 <button class="completeBtn">✓</button>
92 <button class="deleteBtn">✗</button>
93 </div>
94 </div>
95 `;
96
97 $("#taskList").append(taskItem);
98 $("#taskInput").val("");
99
100 updateCounts();
101
102 });
103
104 // Mark as Complete (Event Delegation)
```



This screenshot shows the Visual Studio Code editor with a project named 'GURUNATHRAGANI\_NETFSD\_UPGRAD\_B4\_PYTHON'. The Explorer sidebar on the left shows a file structure with folders for 'WEEK\_1', 'WEEK\_2', 'WEEK\_3', and 'Day1\_handson\_Jquery'. The 'index.html' file is selected. The main editor area displays the following JavaScript code:

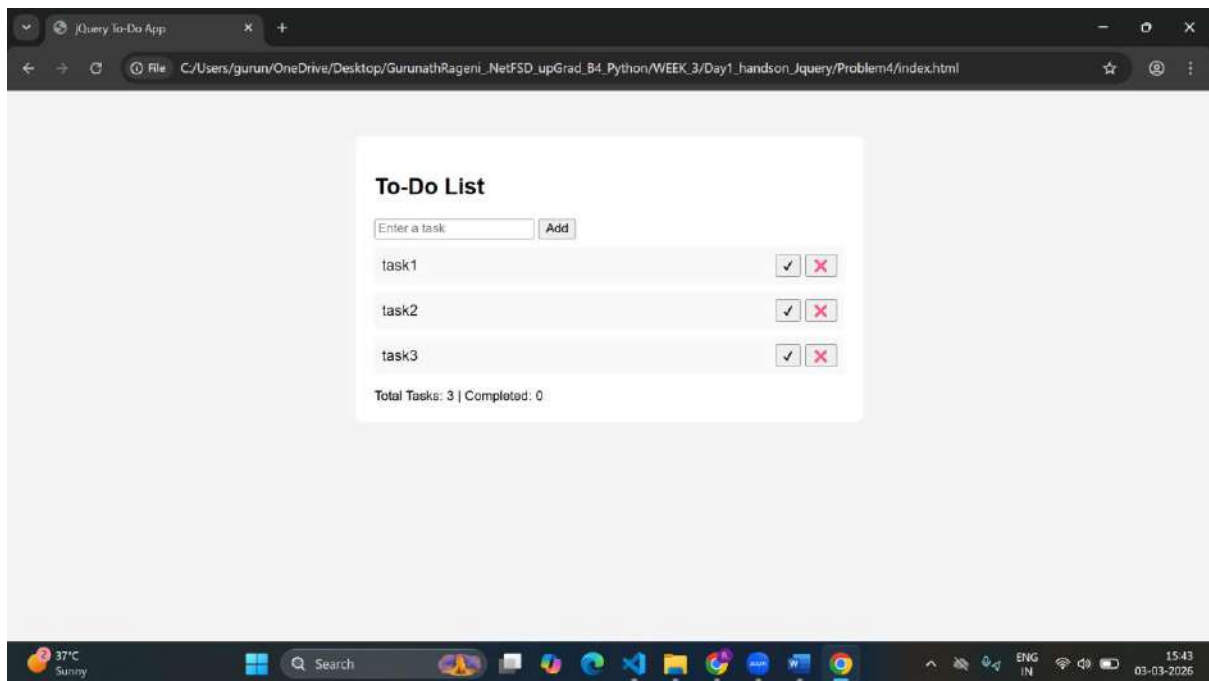
```
WEEK_3 > Day1_handson_Jquery > Problem4 > index.html > html > head > style > container
2  <html>
51 <body>
67 <script>
68 $(document).ready(function () {
103
104 // Mark as Complete (Event Delegation)
105 $("#taskList").on("click", ".completeBtn", function () {
106
107     const taskText = $(this).closest(".task").find(".task-text");
108
109     taskText.toggleClass("completed");
110
111     if (taskText.hasClass("completed")) {
112         completedTasks++;
113     } else {
114         completedTasks--;
115     }
116
117     updateCounts();
118 });
119
120 // Delete Task (Event Delegation)
121 $("#taskList").on("click", ".deleteBtn", function () {
122
123     const taskText = $(this).closest(".task").find(".task-text");
124
125     if (taskText.hasClass("completed")) {
126         completedTasks--;
127     }
```



This screenshot shows the continuation of the JavaScript code in the Visual Studio Code editor. The code is as follows:

```
128     $("#taskList").on("click", ".deleteBtn", function () {
129
130     }
131
132     totalTasks--;
133
134     $(this).closest(".task").remove();
135
136     updateCounts();
137 });
138
139 </script>
140 </body>
141 </html>
```

## Output Screenshot:



## Explanation:

This jQuery To-Do application demonstrates dynamic DOM manipulation by appending task elements using template literals and updating task statistics in real time. It maintains state using counters (`totalTasks`, `completedTasks`) and updates the UI through a dedicated `updateCounts()` function.

Event delegation is implemented on `#taskList` to efficiently handle complete and delete actions for dynamically added tasks. The app uses jQuery methods like `.closest()`, `.find()`, `.toggleClass()`, and `.text()` to manage task states and ensure accurate counter updates.